

Exercise 1**Non-Negative Matrix Factorization (NMF)****1. Projected ALS algorithm**

- (a) We randomly initialize matrices $U^0, V^0 \geq 0$.
- (b) Then, at time-step $t \geq 0$, we alternate between updating U^t and V^t , solving the sub-problems individually. The loss function is

$$\ell(U, V) = \frac{1}{2} \|A - UV\|_F^2$$

which yields the same minimizers U^* when fixing V and V^* when fixing U .

We rewrite the objective as:

$$\ell(U, V) = \frac{1}{2} \|A - UV\|_F^2 = \frac{1}{2} \text{Tr} \left((A - UV)^\top (A - UV) \right)$$

Expanding the terms, we get:

$$\ell(U, V) = \frac{1}{2} \text{Tr}(A^\top A) - \text{Tr}(V^\top U^\top A) + \frac{1}{2} \text{Tr}(V^\top U^\top UV).$$

On the one hand, the gradient w.r.t to U is given by:

$$\nabla_U \ell(U, V) = (UV - A)V^\top$$

On the other hand, the gradient w.r.t to V is given by:

$$\nabla_V \ell(U, V) = U^\top (UV - A).$$

First, we can solve the unconstrained least squares subproblem for V^{t+1} for fixed U^t by setting the gradient to zero:

$$V^{t+1} = \left((U^t)^\top U^t \right)^{-1} (U^t)^\top A$$

Second, we clip the values of the matrix to enforce non-negativity:

$$v_{ij}^{t+1} = \max\{0, v_{ij}^{t+1}\} \quad \forall i, j$$

Third, we can solve the unconstrained least squares subproblem for U^{t+1} for fixed V^{t+1} by setting the gradient to zero:

$$U^{t+1} = AV^{t+1} \left((V^{t+1})^\top V^{t+1} \right)^{-1}$$

Fourth, we clip the values of the matrix to enforce non-negativity:

$$u_{ij}^{t+1} = \max\{0, u_{ij}^{t+1}\} \quad \forall i, j$$

(c) We repeat the previous step until an adequate convergence criterion is reached.

Alternative Derivation: An alternative derivation consists in taking the gradient with respect to each column v_j of V and each row u^i of U individually. Indeed, these columns and rows are independent in the Frobenius norm. In practice, the ALS algorithm is computed this way in order to exploit parallelization. For the **gradient of the loss w.r.t. column vector v_j of V** , we only need to consider the terms depending on v_j :

$$\ell(U, V) = \frac{1}{2} \|A - UV\|_F^2 = \frac{1}{2} \sum_{k=1}^m \|a_k - Uv_k\|_2^2$$

The gradient of the loss w.r.t. column vector v_j of V is hence given by:

$$\nabla_{v_j} \ell(U, V) = U^\top (Uv_j - a_j).$$

By setting the gradient to zero, we obtain that:

$$v_j^{t+1} = \left((U^t)^\top U^t \right)^{-1} (U^t)^\top a_j.$$

Analogously, we only need to consider the terms dependent on the rows u^i when computing the **gradient w.r.t. row vector u^i** :

$$\ell(U, V) = \frac{1}{2} \|A - UV\|_F^2 = \frac{1}{2} \sum_{l=1}^m \|a^l - u^l V\|_2^2.$$

Hence, the gradient becomes:

$$\nabla_{u^i} \ell(U, V) = (u^i V - a^i) V^\top.$$

Setting the gradient to zero, we obtain that:

$$(u^i)^{t+1} = a^i (V^{t+1})^\top \left((V^{t+1})^\top V^{t+1} \right)^{-1}.$$

2. **Coding Exercise** Check the solution Jupyter notebook attached in Moodle.

Exercise 2

Latent Vector Model

- Recall that the log-likelihood function for a sequence of words $\mathbf{x} = x_1, \dots, x_T$ is :

$$\ell(\theta; x) = \sum_{t=1}^T \sum_{\Delta \in \mathcal{I}} \log p(x_{t+\Delta} | x_t; \theta)$$

where $\mathcal{I}$ is a fixed context window.

Suppose that the window size is 3. According to this likelihood, we need to find an embedding $\mathbf{x}_T$ such that

$$\begin{aligned} \arg \max_{\mathbf{x}_T} \ell(\theta; x) &= \arg \max_{\mathbf{x}_T} \sum_{t=1}^T \sum_{\Delta \in \mathcal{I}} \log p(x_{t+\Delta} | x_t; \theta) \\ &= \arg \max_{\mathbf{x}_T} \sum_{t=3}^T \sum_{\Delta \in \mathcal{I}} \log p(\mathbf{x}_T | x_t; \theta) \\ &= \arg \max_{\mathbf{x}_T} \langle \mathbf{x}_{\text{the}}, \mathbf{x}_T \rangle + \langle \mathbf{x}_{\text{on}}, \mathbf{x}_T \rangle + \langle \mathbf{x}_{\text{sat}}, \mathbf{x}_T \rangle \\ &= \arg \max_{\mathbf{x}_T} \langle \mathbf{x}_{\text{the}} + \mathbf{x}_{\text{on}} + \mathbf{x}_{\text{sat}}, \mathbf{x}_T \rangle, \end{aligned}$$

where we only consider terms dependent on x_T . We maximize with respect to the following possible words: $x_T \in \{\text{“horse”}, \text{“mat”}, \text{“chair”}\}$. From the visual embedding diagram provided in the exercise, it is clear that the word “chair” is closest to the summed vector $(\mathbf{x}_{\text{the}} + \mathbf{x}_{\text{on}} + \mathbf{x}_{\text{sat}})$, thus maximizing the above dot product.

You can similarly solve the problem for window size 4.

$$\begin{aligned} \arg \max_{\mathbf{x}_T} \ell(\theta; x) &= \arg \max_{\mathbf{x}_T} \sum_{t=1}^T \sum_{\Delta \in \mathcal{I}} \log p(x_{t+\Delta} | x_t; \theta) \\ &= \arg \max_{\mathbf{x}_T} \sum_{t=2}^T \sum_{\Delta \in \mathcal{I}} \log p(\mathbf{x}_T | x_t; \theta) \\ &= \arg \max_{\mathbf{x}_T} \langle \mathbf{x}_{\text{the}}, \mathbf{x}_T \rangle + \langle \mathbf{x}_{\text{on}}, \mathbf{x}_T \rangle + \langle \mathbf{x}_{\text{sat}}, \mathbf{x}_T \rangle + \langle \mathbf{x}_{\text{cat}}, \mathbf{x}_T \rangle \\ &= \arg \max_{\mathbf{x}_T} \langle (\mathbf{x}_{\text{the}} + \mathbf{x}_{\text{on}} + \mathbf{x}_{\text{sat}} + \mathbf{x}_{\text{cat}}), \mathbf{x}_T \rangle. \end{aligned}$$

From the visual embedding diagram provided in the exercise, it can be seen that “mat” is closest to the summed vector $(\mathbf{x}_{\text{the}} + \mathbf{x}_{\text{on}} + \mathbf{x}_{\text{sat}} + \mathbf{x}_{\text{cat}})$, thus maximizing the above dot product.

Window size	predicted word (horse, mat, or chair)
3	Solution: chair
4	Solution: mat

- We have that $\forall \mathbf{x}_w, \mathbf{x}_v \in \mathbb{R}^d$:

$$\log p(v | w) = \langle \mathbf{x}_w, \mathbf{x}_v \rangle = \langle \mathbf{x}_v, \mathbf{x}_w \rangle = \log p(w | v)$$

since the inner product is symmetric over the real numbers. This implies that $p(v \mid w) = p(w \mid v)$ for all words w, v as the logarithm is an injective function.

Using the definition of the conditional probability (or conditional scores, in this unnormalized case), we obtain that if $p(w), p(v) > 0$, then:

$$p(v \mid w) = \frac{p(v, w)}{p(w)} = \frac{p(v, w)}{p(v)} = p(w \mid v) \iff p(w) = p(v)$$

Thus, we have that the marginal distribution $p(w)$ over the words $w \in \mathcal{V}$ is the uniform distribution. This means that all words are assigned the same prior probability: $p(v) = \frac{1}{n}$ where $n = |\mathcal{V}|$. This is an issue, since in practice certain words are rare whereas others are frequent.

In practice, the normalization constant does partially alleviate this issue. However, the prior distribution still fails to perfectly capture the true empirical distribution. Indeed, the prior distribution is then coupled to the geometry of the embeddings, and the conditional distribution doesn't solely evaluate contextual association.

Two different ways to solve the marginal distribution/scores problem are:

- (a) The first option to solve this issue is to introduce a bias term for the context word v . We ensure that not all the bias terms b_v are the same. We will now prove that the marginal distribution is no longer the uniform distribution by contradiction. For any pair of words (v, w) , we have that:

$$\begin{aligned} p(w) = p(v) &\iff \frac{p(w, v)}{p(v)} = \frac{p(w, v)}{p(w)} \\ &\iff p(w \mid v) = p(v \mid w) \\ &\iff \exp(\langle \mathbf{x}_v, \mathbf{x}_w \rangle + b_w) = \exp(\langle \mathbf{x}_w, \mathbf{x}_v \rangle + b_v) \\ &\iff b_v = b_w. \end{aligned}$$

Since this must hold for all pairs of words (v, w) , it contradicts the assumption that $b_v \neq b_w$ for at least some pair (v, w) .

- (b) The second option is to use different embedding vectors for the conditioned word $v \mapsto \mathbf{y}_v$ and the predicted word $v \mapsto \mathbf{x}_v$. Let us again prove this by contradiction by choosing a random pair of words (v, w) . We have that:

$$\begin{aligned} p(w) = p(v) &\iff \frac{p(w, v)}{p(v)} = \frac{p(w, v)}{p(w)} \\ &\iff p(w \mid v) = p(v \mid w) \\ &\iff \exp(\langle \mathbf{y}_v, \mathbf{x}_w \rangle) = \exp(\langle \mathbf{y}_w, \mathbf{x}_v \rangle) \\ &\iff \langle \mathbf{y}_v, \mathbf{x}_w \rangle = \langle \mathbf{y}_w, \mathbf{x}_v \rangle. \end{aligned}$$

This only holds if all the embeddings for the conditioned word and the predicted word are the same i.e: $\mathbf{y}_v = \mathbf{x}_v, \forall v \in \mathcal{V}$, contradicting our assumption.

3. We define the value N_{vw} :

$$N_{vw} := |\{t : x_t = w, x_{t+\Delta} = v, \Delta \in \mathcal{I}\}|,$$

which counts the occurrences of word v in the context of word w throughout our text. Then, we rewrite the log-likelihood for the skip-gram model in terms of co-occurrences:

$$\ell(\theta; x) = \sum_{t=1}^T \sum_{\Delta \in \mathcal{I}} \log p(x_{t+\Delta} | x_t; \theta) = \sum_{w,v \in \mathcal{V}} N_{vw} \log p(v | w; \theta).$$

Finally, we substitute in the normalized log-bilinear parameterization.

$$\begin{aligned} \ell(\theta; x) &= \sum_{w,v \in \mathcal{V}} N_{vw} \log \left(\frac{\exp(\langle \mathbf{y}_w, \mathbf{x}_v \rangle + b_v)}{\sum_{u \in \mathcal{V}} \exp(\langle \mathbf{y}_w, \mathbf{x}_u \rangle + b_u)} \right) \\ &= \sum_{w,v \in \mathcal{V}} N_{vw} \left[\langle \mathbf{y}_w, \mathbf{x}_v \rangle + b_v - \log \sum_{u \in \mathcal{V}} \exp(\langle \mathbf{y}_w, \mathbf{x}_u \rangle + b_u) \right]. \end{aligned}$$

4. We define the normalization constant $Z_w = \log \sum_{u \in \mathcal{V}} \exp(\langle \mathbf{y}_w, \mathbf{x}_u \rangle + b_u)$. Now, we have that the log-likelihood function can be written as:

$$\ell(\theta; x) = \sum_{w,v \in \mathcal{V}} N_{vw} (\langle \mathbf{y}_w, \mathbf{x}_v \rangle + b_v - Z_w)$$

Hence, the gradient of the log-likelihood function w.r.t a vector $\mathbf{x}_v$ will have to include the gradient of the normalization constant Z_w w.r.t. $\mathbf{x}_v$. However, we have that:

$$\begin{aligned} \nabla_{\mathbf{x}_v} Z_w &= \nabla_{\mathbf{x}_v} \log \left(\sum_{u \in \mathcal{V}} \exp(\langle \mathbf{y}_w, \mathbf{x}_u \rangle + b_u) \right) \\ &= \frac{\exp(\langle \mathbf{y}_w, \mathbf{x}_v \rangle + b_v)}{\sum_{u \in \mathcal{V}} \exp(\langle \mathbf{y}_w, \mathbf{x}_u \rangle + b_u)} \mathbf{y}_w \\ &= p(x_v | x_w; \theta) \mathbf{y}_w. \end{aligned}$$

This gradient includes a summation over the entire vocabulary $\mathcal{V}$. In most cases, the size of the vocabulary $|\mathcal{V}|$ can exceed a million words. Since one has to do this for each parameter update, this sum is prohibitively large if one wants to run the SGD algorithm. It is even more computationally infeasible in the case of gradient descent.

Exercise 3

Negative Sampling

1. By applying the chain rule, we see that for any function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ and $x \in \mathbb{R}^d$:

$$\nabla_x \log \sigma(f(x)) = \frac{1}{\sigma(f(x))} \sigma(f(x)) [1 - \sigma(f(x))] \nabla_x f(x) = [1 - \sigma(f(x))] \nabla_x f(x)$$

where σ is the sigmoid function and $\nabla_x \sigma(x) = \sigma(x)(1 - \sigma(x))$.

Thus, the gradient w.r.t. the parameter $\mathbf{x}_v$ of the negative sampling objective is:

$$\nabla_{\mathbf{x}_v} \ell(\theta; x) = \sum_{w:(w,v) \in S^+} [1 - \sigma(\langle \mathbf{y}_w, \mathbf{x}_v \rangle + b_v)] \mathbf{y}_w - \sum_{w:(w,v) \in S^-} [1 - \sigma(-\langle \mathbf{y}_w, \mathbf{x}_v \rangle - b_v)] \mathbf{y}_w$$

This can be equivalently re-written as:

$$\nabla_{\mathbf{x}_v} \ell(\theta; x) = \sum_{w:(w,v) \in S^+} [1 - \sigma(\langle \mathbf{y}_w, \mathbf{x}_v \rangle + b_v)] \mathbf{y}_w - \sum_{w:(w,v) \in S^-} \sigma(\langle \mathbf{y}_w, \mathbf{x}_v \rangle + b_v) \mathbf{y}_w$$

where we use the fact that $1 - \sigma(-x) = \sigma(x)$.

This gradient only sums over terms explicitly involving the word v (either as a positive or negative context word). Analogously, the gradient with respect to the parameters b_v and $\mathbf{y}_w$ depend only on terms of the sum involving the corresponding words v and w , respectively.

Hence, the gradient descent update rule allows for efficient gradient updates. Instead of the computation complexity depending on the size of the vocabulary $|\mathcal{V}|$, each parameter update only requires computations related to the particular word involved.

In the case of SGD, each update involves only a small sample of positive and negative samples. Additionally, the decomposition of the gradient into independent summation terms naturally supports batch computations, further improving computational efficiency.

2. We define the log-odds h_{vw}^* of the optimal Bayes classifier as:

$$\begin{aligned} h_{vw}^* &= \sigma^{-1}(\mathbb{P}((v, w) = \text{true})) \\ &= \log \frac{\mathbb{P}((v, w) = \text{true})}{1 - \mathbb{P}((v, w) = \text{true})} \\ &= \log \frac{\mathbb{P}((v, w) = \text{true})}{\mathbb{P}((v, w) = \text{false})}, \end{aligned}$$

where we use the fact that $\sigma^{-1}(x) = \log(\frac{y}{1-y})$. Substituting explicitly, we get:

$$\begin{aligned} h_{vw}^* &= \log \frac{\frac{\pi p(v, w)}{\pi p(v, w) + (1 - \pi) q(v, w)}}{\frac{(1 - \pi) q(v, w)}{\pi p(v, w) + (1 - \pi) q(v, w)}} \\ &= \log \frac{\pi p(v, w)}{(1 - \pi) q(v, w)}. \end{aligned}$$

Under the assumption that $\pi = \frac{1}{2}$ and $\alpha = 1$ hence $q(v, w) = p(w)q(v) \propto p(w)p(v)^\alpha = p(w)p(v)$, we obtain that:

$$\begin{aligned}
 h_{vw}^* &= \log \frac{p(v, w)}{q(v, w)} + \log \frac{\pi}{(1 - \pi)} \\
 &= \log \frac{p(v, w)}{q(v, w)} + \log \frac{1/2}{1/2} \\
 &= \log \frac{p(v, w)}{q(v, w)} \\
 &= \log \frac{p(v, w)}{p(w)q(v)} \\
 &= \log \frac{p(v, w)}{p(w)p(v)} \\
 &= \text{PMI}(w; v).
 \end{aligned}$$

Hence, the log-odds h_{vw}^* is equal to the pointwise mutual information between words v and w . In other words, maximizing the log-odds is equivalent to maximizing the odds that words co-occur significantly more often than if they were independent.

Exercise 4

GloVe

1. We first derive the gradient of the loss w.r.t. the embedding vector $\mathbf{x}_v$:

$$\nabla_{\mathbf{x}_v} \ell(\theta; \mathbf{N}) = 2 \sum_{w: N_{vw} > 0} f(N_{vw}) (\log N_{vw} - \langle \mathbf{y}_w, \mathbf{x}_v \rangle) (-\mathbf{y}_w)$$

Analogously, the gradient of the loss w.r.t. $\mathbf{y}_w$ is given by:

$$\nabla_{\mathbf{y}_w} \ell(\theta; \mathbf{N}) = 2 \sum_{v: N_{vw} > 0} f(N_{vw}) (\log N_{vw} - \langle \mathbf{y}_w, \mathbf{x}_v \rangle) (-\mathbf{x}_v)$$

2. Given the above derived gradients, the SGD update for a sampled pair (v, w) with step size η becomes:

$$\begin{aligned} \mathbf{x}_v &\leftarrow \mathbf{x}_v + 2\eta f(N_{vw}) (\log N_{vw} - \langle \mathbf{y}_w, \mathbf{x}_v \rangle) \mathbf{y}_w \\ \mathbf{y}_w &\leftarrow \mathbf{y}_w + 2\eta f(N_{vw}) (\log N_{vw} - \langle \mathbf{y}_w, \mathbf{x}_v \rangle) \mathbf{x}_v \end{aligned}$$

3. The issue with choosing a constant weighting function $f(n) = c, \forall n$ is that it weighs the contribution of all entries (v, w) of the matrix of log-counts $\log \mathbf{N}$ equally. In other words, it assigns the same importance to pairs with low co-occurrence frequencies equally to those with high co-occurrence frequencies.

The main properties that the weighting function f should have are:

- (a) $f(n) \rightarrow 0$ for $n \rightarrow 0$, i.e. the influence of infrequent word-pairs is down-weighted
- (b) $f(n) \rightarrow c$ for $n \rightarrow \infty$, i.e. the influence of frequent word pairs is bounded
- (c) The function f is **monotonically non-decreasing** in general. This implies that frequent co-occurrences are weighed at least as much as infrequent word co-occurrences.

A typical example of a weighing function f is:

$$f(n) = \min \left\{ 1, \left(\frac{n}{n_{\max}} \right)^\alpha \right\}$$

where $\alpha \in (0, 1]$ is a hyperparameter that can be tuned. A typical choice is $\alpha = \frac{3}{4}$.

As shown in Figure 1, the function starts from zero at $N_{ij} = 0$, increases gradually until $n_{\max}$, and remains constant thereafter.

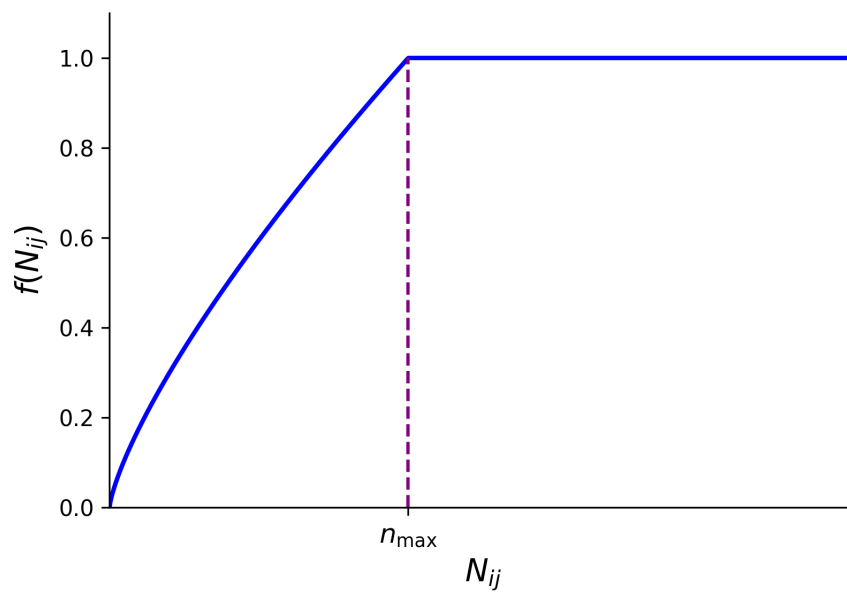


Figure 1: Plot of the typical choice for the weighting function f